Postly v2

Project Documentation

Generated: January 26, 2026

Project Overview

Postly v2 is a social media platform built with Flask (backend) and React (frontend). It features user authentication with JWT, post creation/management, and a responsive feed interface.

Technology Stack

Backend: Flask, Flask-SQLAlchemy, Flask-JWT-Extended
Frontend: React, React Router, Vite
Database: SQLite
Styling: CSS (inline styles in React components)

Project Structure

```
postlyv2/
■■■ backend/
■   ■■■ auth/
■   ■   ■■■ __init__.py
■   ■   ■■■ render.yaml
■   ■   ■■■ routes.py
■   ■   ■■■ utils.py
■   ■■■ database/
■   ■   ■■■ __init__.py
■   ■   ■■■ db.py
■   ■■■ export/
■   ■   ■■■ database/
■   ■   ■   ■■■ __init__.py
■   ■   ■   ■■■ db.py
■   ■   ■■■ project_export.md
■   ■■■ models/
■   ■   ■■■ __init__.py
■   ■   ■■■ post.py
■   ■   ■■■ user.py
■   ■■■ posts/
■   ■   ■■■ routes.py
■   ■■■ static/
■   ■   ■■■ style.css
■   ■■■ templates/
■   ■   ■■■ index.html
■   ■   ■■■ login.html
■   ■   ■■■ posts.html
■   ■   ■■■ register.html
■   ■■■ tools/
■   ■■■ .gitignore
■   ■■■ __init__.py
■   ■■■ app.py
■   ■■■ config.py
■   ■■■ generate_pdf.py
■   ■■■ generate_project_pdf.py
■   ■■■ post.db
■   ■■■ requirements.txt
■■■ frontend/
    ■■■ public/
    ■   ■■■ vite.svg
    ■■■ src/
    ■   ■■■ assets/
    ■   ■   ■■■ react.svg
    ■   ■■■ pages/
    ■   ■   ■■■ Feed.jsx
    ■   ■   ■■■ Login.jsx
    ■   ■   ■■■ Register.jsx
    ■   ■■■ App.css
    ■   ■■■ App.jsx
    ■   ■■■ index.css
    ■   ■■■ main.jsx
    ■■■ .gitignore
    ■■■ README.md
```

```
■■■ eslint.config.js
■■■ index.html
■■■ package-lock.json
■■■ package.json
■■■ vite.config.js
```

Source Code Files

■ backend__init__.py

■ backend\app.py

```
from flask import Flask, render_template, jsonify
from flask_cors import CORS
from flask_jwt_extended import JWTManager
from config import Config
from database.db import db
from auth.routes import auth_bp
from posts.routes import posts_bp

def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)

    # Enable CORS
    CORS(app)

    # Initialize extensions
    db.init_app(app)
    jwt = JWTManager(app)

    # JWT error handlers
    @jwt.expired_token_loader
    def expired_token_callback(jwt_header, jwt_payload):
        return jsonify({"error": "Token has expired"}), 401

    @jwt.invalid_token_loader
    def invalid_token_callback(error):
        return jsonify({"error": f"Invalid token: {str(error)}"}), 401

    @jwt.unauthorized_loader
    def missing_token_callback(error):
        return jsonify({"error": "Authorization header missing"}), 401

    # Register blueprints
    app.register_blueprint(auth_bp, url_prefix="/auth")
    app.register_blueprint(posts_bp, url_prefix="/api/v2/posts")

    with app.app_context():
        db.create_all()
        print("Database and tables created!")

    return app

app = create_app()
```

```
# UI Routes (v1.0 compatibility)
@app.route("/")
def index(): return render_template("index.html")

@app.route("/login-ui")
def login_ui(): return render_template("login.html")

@app.route("/register-ui")
def register_ui(): return render_template("register.html")

@app.route("/posts-ui")
def posts_ui(): return render_template("posts.html")

if __name__ == "__main__":
    app.run(debug=True, port=5000)
```

■ backend\config.py

```
import os
basedir = os.path.abspath(os.path.dirname(__file__))

class Config:
    SECRET_KEY = "super-secret-key"
    JWT_SECRET_KEY = "jwt-secret-key"
    SQLALCHEMY_DATABASE_URI = 'sqlite:///' + os.path.join(basedir, 'post.db')
    SQLALCHEMY_TRACK_MODIFICATIONS = False
```

■ backend\generate_pdf.py

```
#!/usr/bin/env python3
"""Generate a PDF with all project source files"""

from reportlab.lib.pagesizes import letter, A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak, Preformatted, Table, TableStyle
from reportlab.lib import colors
from datetime import datetime
import os

# Define file list with their paths and content
files_data = [
    ("app.py", """from flask import Flask, render_template
from flask_jwt_extended import JWTManager

# Always use package-absolute imports when running as `python -m postly.app`
from config import Config
from database.db import db
from auth.routes import auth_bp
from posts.routes import posts_bp
from models.user import User
from models.post import Post
```

```
def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)

    # Initialize extensions
    db.init_app(app)
    JWTManager(app)

    # Register blueprints
    app.register_blueprint(auth_bp)
    app.register_blueprint(posts_bp)

    # Create tables
    with app.app_context():
        db.create_all()
        print("Database and tables created!")

    return app


app = create_app()

# UI routes
@app.route("/")
def index():
    return render_template("index.html")

@app.route("/login-ui")
def login_ui():
    return render_template("login.html")

@app.route("/register-ui")
def register_ui():
    return render_template("register.html")

@app.route("/posts-ui")
def posts_ui():
    return render_template("posts.html")


if __name__ == "__main__":
    app.run(debug=True)"""),
    ("config.py", """# Configuration settings
import os
basedir = os.path.abspath(os.path.dirname(__file__))

class Config:
    SECRET_KEY = "your-secret-key"
    JWT_SECRET_KEY = "your-jwt-secret-key"
    SQLALCHEMY_DATABASE_URI = 'sqlite:///' + os.path.join(basedir, 'post.db')
    SQLALCHEMY_TRACK_MODIFICATIONS = False"""),
    ("requirements.txt", """# Project dependencies
flask
flask-jwt-extended
flask-sqlalchemy
werkzeug
```

```
python-dotenv
gunicorn"""),
    ("models/user.py", """from postly.database.db import db
from werkzeug.security import generate_password_hash

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False)
    password = db.Column(db.String(200), nullable=False)

    posts = db.relationship("Post", back_populates="user", cascade="all, delete-orphan")"""),
    ("models/post.py", """from postly.database.db import db



class Post(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(100))
    content = db.Column(db.Text)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'))
    user = db.relationship("User", back_populates="posts")"""),
    ("auth/routes.py", """from flask import Blueprint, request, jsonify
from flask_jwt_extended import create_access_token
from werkzeug.security import check_password_hash, generate_password_hash
from database.db import db
from models.user import User

auth_bp = Blueprint("auth", __name__, url_prefix="/auth")


@auth_bp.route("/register", methods=["POST"])
def register():
    data = request.json

    username = data.get("username")
    email = data.get("email")
    password = data.get("password")

    if not username or not email or not password:
        return jsonify({"error": "All fields required"}), 400

    if User.query.filter_by(email=email).first():
        return jsonify({"error": "User already exists"}), 400

    user = User(
        username=username,
        email=email,
        password=generate_password_hash(password)
    )

    db.session.add(user)
    db.session.commit()

    return jsonify({"message": "User registered successfully"}), 201
```

```
@auth_bp.route("/login", methods=["POST"])
def login():
    data = request.json

    email = data.get("email")
    password = data.get("password")

    user = User.query.filter_by(email=email).first()

    if not user or not check_password_hash(user.password, password):
        return jsonify({"error": "Invalid credentials"}), 401

    # ■ IMPORTANT: identity MUST be dict
    access_token = create_access_token(identity={
        "id": user.id,
        "username": user.username
    })

    return jsonify({
        "access_token": access_token,
        "username": user.username
    }), 200"""),
    ("auth/utils.py", """# Authentication utilities
from werkzeug.security import generate_password_hash, check_password_hash

def hash_password(password):
    return generate_password_hash(password)

def verify_password(password, password_hash):
    return check_password_hash(password_hash, password)"""),
    ("posts/routes.py", """from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required, get_jwt_identity
from database.db import db
from models.post import Post

posts_bp = Blueprint("posts", __name__, url_prefix="/posts")


# =========================
# CREATE POST
# =========================
@posts_bp.route("", methods=["POST"])
@jwt_required()
def create_post():
    identity = get_jwt_identity()   # {'id': x, 'username': y}
    user_id = identity["id"]

    data = request.json
    content = data.get("content")

    if not content:
        return jsonify({"error": "Post cannot be empty"}), 400

    post = Post(
        content=content,
        user_id=user_id
```

```
    )

    db.session.add(post)
    db.session.commit()

    return jsonify({"message": "Post created"}), 201

... (290 more lines)
```

■ backend\generate_project_pdf.py

```
"""
Generate a comprehensive PDF with project structure and all code files
"""
import os
import sys
from pathlib import Path
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import inch
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak, Table, TableStyle, Preformatted
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_LEFT

# Get project root
PROJECT_ROOT = Path(__file__).parent.parent

def get_directory_structure(path, prefix="", ignore_dirs={'.git', '__pycache__', 'node_modules', '.venv', 'venv',
    """Generate a text representation of directory structure"""
    items = []
    path = Path(path)

    try:
        entries = sorted(path.iterdir(), key=lambda p: (not p.is_dir(), p.name))
    except PermissionError:
        return items

    for i, entry in enumerate(entries):
        if entry.name.startswith('.') and entry.name not in {'.gitignore', '.env.example'}:
            continue
        if entry.name in ignore_dirs:
            continue

        is_last = i == len(entries) - 1
        current = "■■■ " if is_last else "■■■ "
        next_prefix = "    " if is_last else "■   "

        items.append(prefix + current + entry.name + ("/" if entry.is_dir() else ""))

        if entry.is_dir():
            items.extend(get_directory_structure(entry, prefix + next_prefix, ignore_dirs))

    return items
```

```
def get_code_files(base_path, extensions={'.py', '.jsx', '.js', '.json', '.css', '.html', '.yaml', '.yml', '.txt'
                   ignore_dirs={'.git', '__pycache__', 'node_modules', '.venv', 'venv', 'instance', '.next', 'dis
    """Get all code files in the project"""
    files = []
    base_path = Path(base_path)

    for root, dirs, filenames in os.walk(base_path):
        # Remove ignored directories
        dirs[:] = [d for d in dirs if d not in ignore_dirs and not d.startswith('.')]

        for filename in sorted(filenames):
            if any(filename.endswith(ext) for ext in extensions):
                file_path = Path(root) / filename
                rel_path = file_path.relative_to(PROJECT_ROOT)
                files.append(rel_path)

    return sorted(files)

def read_file_content(file_path, max_lines=500):
    """Read file content with line limit"""
    try:
        with open(file_path, 'r', encoding='utf-8', errors='ignore') as f:
            lines = f.readlines()

        if len(lines) > max_lines:
            content = ''.join(lines[:max_lines])
            content += f"\n... ({len(lines) - max_lines} more lines)\n"
        else:
            content = ''.join(lines)
        return content
    except Exception as e:
        return f"Error reading file: {str(e)}"

def create_pdf():
    """Create the PDF document"""
    output_path = PROJECT_ROOT / "project_documentation.pdf"

    doc = SimpleDocTemplate(
        str(output_path),
        pagesize=letter,
        rightMargin=0.75*inch,
        leftMargin=0.75*inch,
        topMargin=0.75*inch,
        bottomMargin=0.75*inch,
        title="Postly v2 - Project Documentation"
    )

    styles = getSampleStyleSheet()

    # Custom styles
    title_style = ParagraphStyle(
        'CustomTitle',
        parent=styles['Heading1'],
        fontSize=24,
        textColor=colors.HexColor('#1e293b'),
        spaceAfter=12,
```

```
    alignment=TA_CENTER,
    fontName='Helvetica-Bold'
)

heading_style = ParagraphStyle(
    'CustomHeading',
    parent=styles['Heading2'],
    fontSize=16,
    textColor=colors.HexColor('#3b82f6'),
    spaceAfter=10,
    spaceBefore=10,
    fontName='Helvetica-Bold'
)

subheading_style = ParagraphStyle(
    'CustomSubHeading',
    parent=styles['Heading3'],
    fontSize=12,
    textColor=colors.HexColor('#475569'),
    spaceAfter=8,
    spaceBefore=8,
    fontName='Helvetica-Bold'
)

code_style = ParagraphStyle(
    'CodeStyle',
    parent=styles['Normal'],
    fontName='Courier',
    fontSize=8,
    spaceAfter=6,
    leftIndent=12,
    rightIndent=12,
    backColor=colors.HexColor('#f1f5f9'),
    borderPadding=8
)

# Content
content = []

# Title Page
content.append(Spacer(1, 1*inch))
content.append(Paragraph("Postly v2", title_style))
content.append(Paragraph("Project Documentation", styles['Heading2']))
content.append(Spacer(1, 0.3*inch))
content.append(Paragraph(f"Generated: {__import__('datetime').datetime.now().strftime('%B %d, %Y')}", styles[
content.append(Spacer(1, 0.5*inch))

# Project Overview
content.append(Paragraph("Project Overview", heading_style))
content.append(Paragraph(
    "Postly v2 is a social media platform built with Flask (backend) and React (frontend). "
    "It features user authentication with JWT, post creation/management, and a responsive feed interface.",
    styles['Normal']
))
content.append(Spacer(1, 0.3*inch))
```

```
    # Tech Stack
    content.append(Paragraph("Technology Stack", heading_style))
    tech_text = """
    <b>Backend:</b> Flask, Flask-SQLAlchemy, Flask-JWT-Extended<br/>
    <b>Frontend:</b> React, React Router, Vite<br/>
    <b>Database:</b> SQLite<br/>
    <b>Styling:</b> CSS (inline styles in React components)<br/>
    """
    content.append(Paragraph(tech_text, styles['Normal']))
    content.append(Spacer(1, 0.2*inch))

    # Directory Structure
    content.append(PageBreak())
    content.append(Paragraph("Project Structure", heading_style))

    struct_lines = get_directory_structure(PROJECT_ROOT)
    struct_text = "postlyv2/\n" + "\n".join(struct_lines[:100])  # Limit displayed lines

    struct_preformatted = Preformatted(struct_text, code_style)
    content.append(struct_preformatted)
    content.append(Spacer(1, 0.3*inch))

    # Code Files
    content.append(PageBreak())
    content.append(Paragraph("Source Code Files", heading_style))
    content.append(Spacer(1, 0.2*inch))

    # Get and organize files by directory
    code_files = get_code_files(PROJECT_ROOT)

    # Group files by directory
    files_by_dir = {}
    for file_path in code_files:
        dir_name = str(file_path.parent)
        if dir_name not in files_by_dir:
            files_by_dir[dir_name] = []
        files_by_dir[dir_name].append(file_path)

    # Add files for each directory
    for dir_name in sorted(files_by_dir.keys()):
        files = files_by_dir[dir_name]

        for file_path in files:
            content.append(Spacer(1, 0.15*inch))
            content.append(Paragraph(f"■ {file_path}", subheading_style))

... (22 more lines)
```

■ backend\requirements.txt

```
# Project dependencies
flask
flask-jwt-extended
flask-sqlalchemy
```

```
werkzeug
python-dotenv
gunicorn
```

■ backend\auth__init__.py

```
" init "
```

■ backend\auth\render.yaml

```
services:
  - type: web
    name: postly
    env: python
    buildCommand: pip install -r requirements.txt
    startCommand: gunicorn app:app
```

■ backend\auth\routes.py

```
from flask import Blueprint, request, jsonify
from flask_jwt_extended import create_access_token
from werkzeug.security import generate_password_hash, check_password_hash
from database.db import db
from models.user import User

auth_bp = Blueprint("auth", __name__)

@auth_bp.route("/register", methods=["POST"])
def register():
    data = request.get_json()
    if User.query.filter_by(email=data['email']).first():
        return jsonify({"error": "Email already exists"}), 400

    hashed_pw = generate_password_hash(data['password'])
    new_user = User(
        username=data['username'], # Save username
        email=data['email'],
        password_hash=hashed_pw
    )
    db.session.add(new_user)
    db.session.commit()
    return jsonify({"message": "User created"}), 201

@auth_bp.route("/login", methods=["POST"])
def login():
    data = request.get_json()
    user = User.query.filter_by(email=data['email']).first()

    if user and check_password_hash(user.password_hash, data['password']):
        # We store both ID and Username in the token identity for easy access
```

```
        access_token = create_access_token(identity={"id": user.id, "username": user.username})
        return jsonify({
            "token": access_token,
            "userId": user.id,
            "username": user.username
        }), 200

    return jsonify({"error": "Invalid credentials"}), 401
```

■ backend\auth\utils.py

```
# Authentication utilities
from werkzeug.security import generate_password_hash, check_password_hash

def hash_password(password):
    return generate_password_hash(password)

def verify_password(password, password_hash):
    return check_password_hash(password_hash, password)
```

■ backend\database__init__.py

```
" init "
```

■ backend\database\db.py

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()
```

■ backend\export\database__init__.py

```
" init "
```

■ backend\export\database\db.py

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()
```

■ backend\models__init__.py

```
"init"
```

■ backend\models\post.py

```
from database.db import db
from datetime import datetime

class Post(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    content = db.Column(db.Text, nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow) # New Field

    # Relationship to get the username easily
    author = db.relationship('User', backref=db.backref('posts', lazy=True))
```

■ backend\models\user.py

```
from database.db import db
from werkzeug.security import generate_password_hash, check_password_hash

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False) # New Field
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(128), nullable=False)

    # ADD THIS LINE: This tells SQLAlchemy to link to the Post model
    posts = db.relationship('Post', back_populates='author', lazy=True)

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)
```

■ backend\posts\routes.py

```
from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required, get_jwt_identity
from database.db import db
from models.post import Post

# The prefix is now handled in app.py registration
posts_bp = Blueprint("posts", __name__)

def get_current_user_id():
    """Helper to extract user ID regardless of JWT identity format"""
    identity = get_jwt_identity()
    if isinstance(identity, dict):
        return identity.get("id")
    return int(identity)
```

```
@posts_bp.route("", methods=["GET"])
def get_posts():
    # Join Post and User tables to get the email/username
    posts_query = db.session.query(Post, User).join(User, Post.user_id == User.id).order_by(Post.id.desc()).all()

    result = []
    for post, user in posts_query:
        result.append({
            "id": post.id,
            "content": post.content,
            "user_id": post.user_id,
            "username": user.email.split('@')[0], # Temporary username from email
            "created_at": post.id # We'll add real timestamps next
        })

    return jsonify(result), 200

# CREATE POST
@posts_bp.route("", methods=['POST'])
@jwt_required()
def create_post():
    try:
        user_id = get_current_user_id()
        data = request.get_json()

        if not data or 'content' not in data or not data['content'].strip():
            return jsonify({"error": "Content is required"}), 400

        new_post = Post(content=data['content'], user_id=user_id)
        db.session.add(new_post)
        db.session.commit()
        return jsonify({"message": "Post created!", "id": new_post.id}), 201
    except Exception as e:
        print(f"Error creating post: {str(e)}")
        return jsonify({"error": str(e)}), 500

# DELETE & UPDATE (Under "/<id>")
@posts_bp.route("/<int:post_id>", methods=["PUT", "DELETE"])
@jwt_required()
def modify_post(post_id):
    user_id = get_current_user_id()
    post = Post.query.get_or_404(post_id)

    if post.user_id != user_id:
        return jsonify({"error": "Unauthorized"}), 403

    if request.method == "DELETE":
        db.session.delete(post)
        db.session.commit()
        return jsonify({"message": "Post deleted"}), 200

    if request.method == "PUT":
        data = request.get_json()
        new_content = data.get("content")
        if not new_content:
            return jsonify({"error": "Content cannot be empty"}), 400
```

```
        post.content = new_content
        db.session.commit()
        return jsonify({"message": "Post updated"}), 200

@posts_bp.route("", methods=['GET', 'POST'])
def handle_posts():
    if request.method == 'GET':
        posts_query = Post.query.order_by(Post.id.desc()).all()
        return jsonify([{
            "id": p.id,
            "content": p.content,
            "user_id": p.user_id,
            "username": p.author.username, # Sending username
            "created_at": p.created_at.strftime("%b %d, %H:%M") # Formatting time
        } for p in posts_query]), 200
    # ... keep POST logic same as before ...
```

■ backend\static\style.css

```
/* Modern Professional Theme */
:root {
    --primary: #4f46e5;
    --primary-hover: #4338ca;
    --bg: #f3f4f6;
    --white: #ffffff;
    --text-dark: #111827;
    --text-light: #6b7280;
    --border: #d1d5db;
    --shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
}

body {
    font-family: 'Inter', system-ui, sans-serif;
    background-color: var(--bg);
    margin: 0;
    display: flex;
    align-items: center;
    justify-content: center;
    min-height: 100vh;
}

.container {
    background: var(--white);
    padding: 2.5rem;
    border-radius: 1rem;
    box-shadow: var(--shadow);
    width: 100%;
    max-width: 400px;
    text-align: center;
}

h2 {
    color: var(--text-dark);
    margin-bottom: 0.5rem;
```

```
    font-size: 1.8rem;
}

p.subtitle {
    color: var(--text-light);
    margin-bottom: 2rem;
    font-size: 0.9rem;
}

/* Form Styles */
input {
    width: 100%;
    padding: 0.75rem 1rem;
    margin-bottom: 1rem;
    border: 1px solid var(--border);
    border-radius: 0.5rem;
    box-sizing: border-box;
    font-size: 1rem;
    transition: all 0.2s;
}

input:focus {
    outline: none;
    border-color: var(--primary);
    box-shadow: 0 0 0 3px rgba(79, 70, 229, 0.1);
}

button {
    width: 100%;
    padding: 0.75rem;
    background-color: var(--primary);
    color: white;
    border: none;
    border-radius: 0.5rem;
    font-size: 1rem;
    font-weight: 600;
    cursor: pointer;
    transition: background 0.2s;
}

button:hover {
    background-color: var(--primary-hover);
}

.link-text {
    margin-top: 1.5rem;
    font-size: 0.875rem;
    color: var(--text-light);
}

.link-text a {
    color: var(--primary);
    text-decoration: none;
    font-weight: 500;
}
```

```
/* Feed specific article styling */
article {
    text-align: left;
    background: #f9fafb;
    padding: 1rem;
    border-radius: 0.5rem;
    margin-bottom: 1rem;
    border-left: 4px solid var(--primary);
}
```

■ backend\templates\index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Postly</title>
</head>
<body>
<div class="container">
    <h2>Welcome to Postly</h2>
    <button onclick="window.location.href='/register-ui'" style="margin-bottom: 10px;">Register</button>
    <button onclick="window.location.href='/login-ui'" style="background: #fff; color: #4f46e5; border: 1px solid
</div>
</body>
</html>
```

■ backend\templates\login.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Login</title>
</head>
<body>
<div class="container">
    <h2>Welcome Back</h2>
    <p class="subtitle">Log in to your Postly account</p>

    <input type="email" id="email" placeholder="Email Address">
    <input type="password" id="password" placeholder="Password">

    <button id="login-btn">Login</button>

    <p class="link-text">
        Don't have an account? <a href="/register-ui">Sign up</a>
    </p>
    <div id="msg"></div>
</div>

<script>
```

```
    document.getElementById("login-btn").addEventListener("click", async () => {
        const email = document.getElementById("email").value;
        const password = document.getElementById("password").value;

        const res = await fetch("/auth/login", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ email, password })
        });

        const data = await res.json();
        if (res.ok) {
            localStorage.setItem("token", data.access_token);
            window.location.href = "/posts-ui";
        } else {
            document.getElementById("msg").innerText = data.error || "Login failed";
            document.getElementById("msg").style.color = "red";
        }
    });
</script>
</body>
</html>
```

■ backend\templates\posts.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Postly - Feed</title>
    <link rel="stylesheet" href="/static/style.css">
    <style>
        .demo-banner {
            background: #fffbeb;
            border: 1px solid #fcd34d;
            color: #92400e;
            padding: 15px;
            border-radius: 8px;
            margin-bottom: 20px;
            font-size: 13px;
            text-align: left;
        }
        .footer {
            margin-top: 40px;
            padding-top: 20px;
            border-top: 1px solid #e2e8f0;
            text-align: center;
            font-size: 14px;
            color: #64748b;
        }
        .social-links {
            margin-top: 10px;
            display: flex;
            justify-content: center;
            gap: 15px;
```

```
        }
        .social-links a {
            color: #4f46e5;
            text-decoration: none;
            font-weight: 600;
        }
        .social-links a:hover {
            text-decoration: underline;
        }
    </style>
</head>
<body>
    <div class="container">
        <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 10px;">
            <h2>Postly Feed</h2>
            <button id="logout-btn" style="width: auto; padding: 5px 15px; background: #64748b;">Logout</button>
        </div>

        <div class="demo-banner">
            <strong>■ Demo Warning:</strong> This is a demo project deployed for learning and portfolio purposes
        </div>

        <hr>

        <h2>Create Post</h2>
        <textarea id="content" placeholder="What's on your mind?" rows="4" style="width: 100%;"></textarea>
        <br><br>
        <button id="post-btn">Post to Feed</button>

        <hr>

        <h2>Recent Posts</h2>
        <div id="feed"></div>

        <div class="footer">
            <p>■ Built by <strong>Kervin Kittuselvan</strong> — an IT engineering student</p>
            <div class="social-links">
                <a href="https://www.linkedin.com/in/kervin-kittuselvan-6b2927256" target="_blank">LinkedIn</a>
                <a href="https://github.com/Kervin45" target="_blank">GitHub</a>
                <a href="https://www.instagram.com/its.me_kervin?igsh=MTU2Mm0xN283ZHM1Yw==" target="_blank">Insta
            </div>
        </div>
    </div>

    <script>
        const token = localStorage.getItem("token");

        // 1. Protection: Redirect if not logged in
        if (!token) {
            window.location.href = "/login-ui";
        }

        // 2. CREATE: Add new post
        async function createPost() {
            const content = document.getElementById("content").value;
            if (!content) {
```

```
        alert("Post cannot be empty");
        return;
    }

    try {
        const res = await fetch("/posts", {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
                "Authorization": "Bearer " + token
            },
            body: JSON.stringify({ content })
        });

        if (res.ok) {
            document.getElementById("content").value = "";
            loadPosts();
        } else {
            const errorData = await res.json();
            alert("Post failed: " + (errorData.error || errorData.msg || "Unknown error"));
        }
    } catch (err) {
        alert("Server connection error");
    }
}

// 3. READ: Load all posts
async function loadPosts() {
    try {
        const res = await fetch("/posts");
        const posts = await res.json();
        const feed = document.getElementById("feed");
        feed.innerHTML = "";

        posts.forEach(p => {
            const article = document.createElement("article");
            article.innerHTML = `
                <p id="text-${p.id}">${p.content}</p>
                <small>Posted by User #${p.user_id}</small>
                <div style="margin-top: 12px; display: flex; gap: 10px;">
                    <button onclick="editPost(${p.id}, '${p.content.replace(/'/g, "\\'")}')" style="width
                    <button onclick="deletePost(${p.id})" style="width: auto; padding: 5px 12px; backgrou
                </div>
            `;
            feed.appendChild(article);
        });
    } catch (err) {
        console.error("Error loading feed:", err);
    }
}

// 4. DELETE: Remove a post
async function deletePost(id) {
    if (!confirm("Are you sure you want to delete this post?")) return;

    try {
```

```
            const res = await fetch(`/posts/${id}`, {
                method: "DELETE",
                headers: {
                    "Authorization": "Bearer " + token
                }
            });

            if (res.ok) {
                loadPosts();
            } else {
                alert("Unauthorized: You can only delete your own posts.");
            }
        } catch (err) {
            alert("Error deleting post");
        }
    }

    // 5. UPDATE: Edit a post
    async function editPost(id, currentContent) {
        const newContent = prompt("Edit your post:", currentContent);

        if (newContent === null || newContent === currentContent || newContent.trim() === "") {
            return;
        }

        try {
            const res = await fetch(`/posts/${id}`, {
                method: "PUT",
                headers: {
                    "Content-Type": "application/json",
                    "Authorization": "Bearer " + token
                },
                body: JSON.stringify({ content: newContent })
            });

            if (res.ok) {
                loadPosts();
            } else {
                alert("Unauthorized: You can only edit your own posts.");
            }
        } catch (err) {
            alert("Error updating post");
        }
    }

    // Event Listeners
    document.getElementById("post-btn").addEventListener("click", createPost);

    document.getElementById("logout-btn").addEventListener("click", () => {
        localStorage.removeItem("token");
        window.location.href = "/login-ui";
    });

    // Initial load
    loadPosts();
</script>
```

```
</body>
</html>
```

■ backend\templates\register.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Register</title>
</head>
<body>
<div class="container">
    <h2>Create Account</h2>
    <p class="subtitle">Join Postly and start sharing</p>

    <input type="text" id="username" placeholder="Username">
    <input type="email" id="email" placeholder="Email Address">
    <input type="password" id="password" placeholder="Password">

    <button id="reg-btn">Register</button>

    <p class="link-text">
        Already have an account? <a href="/login-ui">Login here</a>
    </p>
    <div id="msg"></div>
</div>

<script>
    document.getElementById("reg-btn").addEventListener("click", async () => {
        const username = document.getElementById("username").value;
        const email = document.getElementById("email").value;
        const password = document.getElementById("password").value;

        const res = await fetch("/auth/register", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ username, email, password })
        });

        const data = await res.json();
        if (res.ok) {
            alert("Registration successful! Please login.");
            window.location.href = "/login-ui";
        } else {
            document.getElementById("msg").innerText = data.error || "Registration failed";
            document.getElementById("msg").style.color = "red";
        }
    });
</script>
</body>
</html>
```

■ frontend\eslint.config.js

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
      parserOptions: {
        ecmaVersion: 'latest',
        ecmaFeatures: { jsx: true },
        sourceType: 'module',
      },
    },
    rules: {
      'no-unused-vars': ['error', { varsIgnorePattern: '^[A-Z_]' }],
    },
  },
])
```

■ frontend\index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>frontend</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

■ frontend\package-lock.json

```
{
```

```
"name": "frontend",
"version": "0.0.0",
"lockfileVersion": 3,
"requires": true,
"packages": {
  "": {
    "name": "frontend",
    "version": "0.0.0",
    "dependencies": {
      "react": "^19.2.0",
      "react-dom": "^19.2.0",
      "react-router-dom": "^7.13.0"
    },
    "devDependencies": {
      "@eslint/js": "^9.39.1",
      "@types/react": "^19.2.5",
      "@types/react-dom": "^19.2.3",
      "@vitejs/plugin-react": "^5.1.1",
      "eslint": "^9.39.1",
      "eslint-plugin-react-hooks": "^7.0.1",
      "eslint-plugin-react-refresh": "^0.4.24",
      "globals": "^16.5.0",
      "vite": "^7.2.4"
    }
  },
  "node_modules/@babel/code-frame": {
    "version": "7.28.6",
    "resolved": "https://registry.npmjs.org/@babel/code-frame/-/code-frame-7.28.6.tgz",
    "integrity": "sha512-JYgintcMjRiCvS8mMECzaEn+m3PfoQiyqukOMCCVQtoJGYJw8j/8LBJEiqkHLkfwCcs74E3pbAUFNg7d9VNJ+Q
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "@babel/helper-validator-identifier": "^7.28.5",
      "js-tokens": "^4.0.0",
      "picocolors": "^1.1.1"
    },
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/compat-data": {
    "version": "7.28.6",
    "resolved": "https://registry.npmjs.org/@babel/compat-data/-/compat-data-7.28.6.tgz",
    "integrity": "sha512-2lfu57JtzctfIrcGMz992hyLlByuzgIk58+hhGCxjKZ3rWI82NnVLjXcaTqkI2NvlcvOskZaiZ5kjUALo3Lpxg
    "dev": true,
    "license": "MIT",
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/core": {
    "version": "7.28.6",
    "resolved": "https://registry.npmjs.org/@babel/core/-/core-7.28.6.tgz",
    "integrity": "sha512-H3mcG6ZDLTlYfaSNi0iOKkigqMFvkTKlGUYlD8GW7nNOYRrevuA46iTypPyv+06V3fEmvvazfntkBU34L0azAw
    "dev": true,
    "license": "MIT",
```

```
    "peer": true,
    "dependencies": {
      "@babel/code-frame": "^7.28.6",
      "@babel/generator": "^7.28.6",
      "@babel/helper-compilation-targets": "^7.28.6",
      "@babel/helper-module-transforms": "^7.28.6",
      "@babel/helpers": "^7.28.6",
      "@babel/parser": "^7.28.6",
      "@babel/template": "^7.28.6",
      "@babel/traverse": "^7.28.6",
      "@babel/types": "^7.28.6",
      "@jridgewell/remapping": "^2.3.5",
      "convert-source-map": "^2.0.0",
      "debug": "^4.1.0",
      "gensync": "^1.0.0-beta.2",
      "json5": "^2.2.3",
      "semver": "^6.3.1"
    },
    "engines": {
      "node": ">=6.9.0"
    },
    "funding": {
      "type": "opencollective",
      "url": "https://opencollective.com/babel"
    }
  },
  "node_modules/@babel/generator": {
    "version": "7.28.6",
    "resolved": "https://registry.npmjs.org/@babel/generator/-/generator-7.28.6.tgz",
    "integrity": "sha512-lOoVRwADj8hjf7al89tvQ2a1lf53Z+7tiXMgpZJL3maQPDxh0DgLMN62B2MKUOFcoodBHLMbDM6WAbKgNy5Suw
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "@babel/parser": "^7.28.6",
      "@babel/types": "^7.28.6",
      "@jridgewell/gen-mapping": "^0.3.12",
      "@jridgewell/trace-mapping": "^0.3.28",
      "jsesc": "^3.0.2"
    },
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/helper-compilation-targets": {
    "version": "7.28.6",
    "resolved": "https://registry.npmjs.org/@babel/helper-compilation-targets/-/helper-compilation-targets-7.28
    "integrity": "sha512-JYtls3hqi15fcx5GaSNL7SCTJ2MNmjrkHXg4FSpOA/grxK8KwyZ5bubHsCq8FXCkua6xhuaaBit+3b7+VZRfcA
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "@babel/compat-data": "^7.28.6",
      "@babel/helper-validator-option": "^7.27.1",
      "browserslist": "^4.24.0",
      "lru-cache": "^5.1.1",
      "semver": "^6.3.1"
    },
```

```
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/helper-globals": {
    "version": "7.28.0",
    "resolved": "https://registry.npmjs.org/@babel/helper-globals/-/helper-globals-7.28.0.tgz",
    "integrity": "sha512-+W6cISkXFa1jXsDEdYA8HeevQT/FULhxzR99pxphltZcVaugps53THCeiWA8SguxxpSp3gKPiuYfSWopkLQ4hw
    "dev": true,
    "license": "MIT",
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/helper-module-imports": {
    "version": "7.28.6",
    "resolved": "https://registry.npmjs.org/@babel/helper-module-imports/-/helper-module-imports-7.28.6.tgz",
    "integrity": "sha512-l5XkZK7r7wa9LucGw9LwZyyCUscb4x37JWTPz7swwFE/0FMQAGpiWUZn8u9DzkSBWEcK25jmvubfpw2dnAMdbw
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "@babel/traverse": "^7.28.6",
      "@babel/types": "^7.28.6"
    },
    "engines": {
      "node": ">=6.9.0"
    }
  },
  "node_modules/@babel/helper-module-transforms": {
    "version": "7.28.6",
    "resolved": "https://registry.npmjs.org/@babel/helper-module-transforms/-/helper-module-transforms-7.28.6.t
    "integrity": "sha512-67oXFAYr2cDLDVGLXTEABjdBJZ6drElUSI7WKp70NrpyISso3plG9SAGEF6y7zbha/wOzUByWWTJvEDVNIUGcA
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "@babel/helper-module-imports": "^7.28.6",
      "@babel/helper-validator-identifier": "^7.28.5",
      "@babel/traverse": "^7.28.6"
    },
    "engines": {
      "node": ">=6.9.0"
    },
    "peerDependencies": {
      "@babel/core": "^7.0.0"
    }
  },
  "node_modules/@babel/helper-plugin-utils": {
    "version": "7.28.6",
    "resolved": "https://registry.npmjs.org/@babel/helper-plugin-utils/-/helper-plugin-utils-7.28.6.tgz",
    "integrity": "sha512-S9gzZ/bz83GRysI7gAD4wPT/AI3uCnY+9xn+Mx/KPs2JwHJIz1W8PZkg2cqyt3RNOBM8ejcXhV6y8Og7ly/Dug
    "dev": true,
    "license": "MIT",
    "engines": {
      "node": ">=6.9.0"
    }
  },
```

```
    "node_modules/@babel/helper-string-parser": {
      "version": "7.27.1",
      "resolved": "https://registry.npmjs.org/@babel/helper-string-parser/-/helper-string-parser-7.27.1.tgz",
      "integrity": "sha512-qMlSxKbpRlAridDExk92nSobyDdpPijUq2DW6oDnUqd0iOGxmQjyqhMIihI9+zv4LPyZdRje2cavWPbCbWm3eA
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helper-validator-identifier": {
      "version": "7.28.5",
      "resolved": "https://registry.npmjs.org/@babel/helper-validator-identifier/-/helper-validator-identifier-7.
      "integrity": "sha512-qSs4ifwzKJSV39ucNjsvc6WVHs6b7S03sOh2OcHF9UHfVPqWWALUsNUVzhSBiItjRZoLHx7nIarVjqKVusUZ1Q
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helper-validator-option": {
      "version": "7.27.1",
      "resolved": "https://registry.npmjs.org/@babel/helper-validator-option/-/helper-validator-option-7.27.1.tgz
      "integrity": "sha512-YvjJow9FxbhFFKDSuFnVCe2WxXk1zWc22fFePVNEaWJEu8IrZVlda6N0uHwzZrUM1il7NC9Mlp4MaJYbYd9JSg
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6.9.0"
      }
    },
    "node_modules/@babel/helpers": {

... (2785 more lines)
```

■ frontend\package.json

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-router-dom": "^7.13.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
```

```
    "@types/react": "^19.2.5",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^5.1.1",
    "eslint": "^9.39.1",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "vite": "^7.2.4"
  }
}
```

■ frontend\vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

■ frontend\src\App.css

```
.App {
  max-width: 600px;
  margin: 0 auto;
  padding: 2rem;
  font-family: sans-serif;
  color: #333;
}

.post-card {
  background: #f9f9f9;
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 15px;
  margin-bottom: 15px;
  text-align: left;
}

.post-card p {
  font-size: 1.1rem;
  margin: 0 0 10px 0;
}

.post-card small {
  color: #666;
}
```

■ frontend\src\App.jsx

```
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Login from './pages/Login';
import Register from './pages/Register'; // Import this!
import Feed from './pages/Feed';

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/feed" element={<Feed />} />
      </Routes>
    </Router>
  );
}

export default App;
```

■ frontend\src\index.css

```
:root {
  background-color: #ffffff;
  color: #213547;
}

body {
  margin: 0;
  display: flex;
  place-items: flex-start;
  min-width: 320px;
  min-height: 100vh;
  font-family: Inter, system-ui, Avenir, Helvetica, Arial, sans-serif;
}

#root {
  width: 100%;
}
```

■ frontend\src\main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

■ frontend\src\pages\Feed.jsx

```
import { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';

function Feed() {
    const [posts, setPosts] = useState([]);
    const [newPost, setNewPost] = useState("");
    const [loading, setLoading] = useState(false);
    const navigate = useNavigate();

    // Get stored data
    const token = localStorage.getItem("token");
    const currentUserId = parseInt(localStorage.getItem("userId"));

    const fetchPosts = async () => {
        setLoading(true);
        try {
            const res = await fetch("http://127.0.0.1:5000/api/v2/posts");
            const data = await res.json();
            setPosts(data);
        } catch (err) {
            console.error("Failed to fetch");
        } finally {
            setLoading(false);
        }
    };

    useEffect(() => {
        if (!token) { navigate("/"); return; }
        fetchPosts();
    }, []);

    const handleCreatePost = async (e) => {
        if (e) e.preventDefault(); // For Enter key support
        if (!newPost.trim()) return;

        setLoading(true);
        const response = await fetch("http://127.0.0.1:5000/api/v2/posts", {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
                "Authorization": `Bearer ${token}`
            },
            body: JSON.stringify({ content: newPost })
        });

        if (response.ok) {
            setNewPost("");
            fetchPosts();
        } else {
            const errData = await response.json();
            if (response.status === 401) logout();
            alert(errData.error || "Post failed");
```

```
        }
        setLoading(false);
    };

    const logout = () => {
        localStorage.clear();
        navigate("/");
    };

    return (
        <div style={styles.container}>
            <header style={styles.header}>
                <h2 style={{margin:0}}>Postly ■</h2>
                <button onClick={logout} style={styles.logoutBtn}>Logout</button>
            </header>

            {/* Create Post Area */}
            <form onSubmit={handleCreatePost} style={styles.postBox}>
                <textarea
                    value={newPost}
                    onChange={(e) => setNewPost(e.target.value)}
                    onKeyDown={(e) => e.key === 'Enter' && !e.shiftKey && handleCreatePost(e)}
                    placeholder="What's happening?"
                    style={styles.textarea}
                />
                <button type="submit" disabled={loading} style={styles.postBtn}>
                    {loading ? "Posting..." : "Post"}
                </button>
            </form>

            {/* Feed */}
            <div style={styles.feed}>
                {loading && posts.length === 0 && <p>Loading feed...</p>}
                {posts.map(p => (
                    <div key={p.id} style={styles.card}>
                        <div style={styles.cardHeader}>
                            <span style={styles.username}>@{p.username}</span>
                            <span style={styles.time}>{p.created_at}</span>
                        </div>
                        <p style={styles.content}>{p.content}</p>

                        {/* ONLY show buttons if it's YOUR post */}
                        {p.user_id === currentUserId && (
                            <div style={styles.actions}>
                                <button onClick={() => {/* handleEdit code */}} style={styles.editBtn}>Edit</butt
                                <button onClick={() => {/* handleDelete code */}} style={styles.deleteBtn}>Delete
                            </div>
                        )}
                    </div>
                ))}
            </div>
        </div>
    );
}

// Pro UI Styles
```

```
const styles = {
    container: { maxWidth: '600px', margin: '0 auto', fontFamily: 'system-ui, sans-serif', backgroundColor: '#f8f
    header: { display: 'flex', justifyContent: 'space-between', alignItems: 'center', padding: '20px', backgroun
    postBox: { padding: '20px', backgroundColor: '#fff', borderBottom: '8px solid #f1f5f9' },
    textarea: { width: '100%', border: '1px solid #e2e8f0', borderRadius: '12px', padding: '15px', fontSize: '16p
    postBtn: { marginTop: '10px', padding: '10px 24px', backgroundColor: '#3b82f6', color: '#fff', border: 'none'
    card: { padding: '20px', backgroundColor: '#fff', borderBottom: '1px solid #e2e8f0' },
    cardHeader: { display: 'flex', gap: '10px', marginBottom: '8px', fontSize: '14px' },
    username: { fontWeight: '700', color: '#1e293b' },
    time: { color: '#64748b' },
    content: { fontSize: '16px', color: '#334155', lineHeight: '1.5', margin: 0 },
    logoutBtn: { backgroundColor: '#f1f5f9', border: 'none', padding: '8px 16px', borderRadius: '8px', cursor: 'p
    actions: { marginTop: '12px', display: 'flex', gap: '15px' },
    editBtn: { background: 'none', border: 'none', color: '#6366f1', cursor: 'pointer', fontSize: '13px', fontWei
    deleteBtn: { background: 'none', border: 'none', color: '#ef4444', cursor: 'pointer', fontSize: '13px', fontW
};

export default Feed;
```

■ frontend\src\pages\Login.jsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';

function Login() {
    const [formData, setFormData] = useState({ email: '', password: '' });
    const navigate = useNavigate();

    const handleLogin = async (e) => {
        e.preventDefault();
        const res = await fetch("http://127.0.0.1:5000/auth/login", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(formData)
        });
        const data = await res.json();
        if (res.ok) {
            localStorage.setItem("token", data.token);
            localStorage.setItem("userId", data.userId); // Store this for the feed logic
            localStorage.setItem("username", data.username);
            navigate("/feed");
        } else {
            alert("Invalid credentials");
        }
    };

    return (
        <div style={styles.container}>
            <form onSubmit={handleLogin} style={styles.card}>
                <h2 style={styles.title}>Welcome Back</h2>
                <input type="email" placeholder="Email" required style={styles.input}
                    onChange={(e) => setFormData({...formData, email: e.target.value})} />
                <input type="password" placeholder="Password" required style={styles.input}
                    onChange={(e) => setFormData({...formData, password: e.target.value})} />
```

```
                <button type="submit" style={styles.button}>Login</button>
                <p onClick={() => navigate("/register")} style={styles.link}>New here? Register</p>
            </form>
        </div>
    );
}

// (Use the same 'styles' object as Register.js)
export default Login;
```

■ frontend\src\pages\Register.jsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';

function Register() {
    const [formData, setFormData] = useState({ username: '', email: '', password: '' });
    const navigate = useNavigate();

    const handleRegister = async (e) => {
        e.preventDefault();
        const res = await fetch("http://127.0.0.1:5000/auth/register", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(formData)
        });
        if (res.ok) {
            alert("Account created! Log in now.");
            navigate("/");
        } else {
            alert("Registration failed. Email or Username might be taken.");
        }
    };

    return (
        <div style={styles.container}>
            <form onSubmit={handleRegister} style={styles.card}>
                <h2 style={styles.title}>Join Postly ■</h2>
                <input type="text" placeholder="Username" required style={styles.input}
                    onChange={(e) => setFormData({...formData, username: e.target.value})} />
                <input type="email" placeholder="Email" required style={styles.input}
                    onChange={(e) => setFormData({...formData, email: e.target.value})} />
                <input type="password" placeholder="Password" required style={styles.input}
                    onChange={(e) => setFormData({...formData, password: e.target.value})} />
                <button type="submit" style={styles.button}>Create Account</button>
                <p onClick={() => navigate("/")} style={styles.link}>Already have an account? Login</p>
            </form>
        </div>
    );
}

const styles = {
    container: { display: 'flex', justifyContent: 'center', alignItems: 'center', height: '100vh', backgroundColo
    card: { backgroundColor: '#fff', padding: '40px', borderRadius: '16px', boxShadow: '0 4px 6px rgba(0,0,0,0.1)
```

```
    title: { textAlign: 'center', color: '#1e293b', marginBottom: '10px' },
    input: { padding: '12px', borderRadius: '8px', border: '1px solid #e2e8f0', outline: 'none' },
    button: { padding: '12px', borderRadius: '8px', border: 'none', backgroundColor: '#3b82f6', color: '#fff', fo
    link: { textAlign: 'center', fontSize: '14px', color: '#6366f1', cursor: 'pointer' }
};

export default Register;
```